

EEE3096S Practical 1B

1st Ebrahim Bhyat
BHYEBR002

2nd Joshua Smith
SMTJOS022

I. INTRODUCTION

The aim of this practical is to profile the computational performance of the STM32F0 development board through the execution of the Mandelbrot set algorithm. In order to compare the run times and accuracy, two functions are used one being the fixed point arithmetic and the other floating-point arithmetic. This will be done to measure the performance, gain greater depth into the impact of different data types on speed, and practice the use of STM32CubeIDE debugger and live expressions. Furthermore, running the algorithms on both Python and personal computers (PC's) allows one to gauge the performance of the STM32F0.

Additional materials for this report can be found on GitHub at the [link](#) :

II. METHODOLOGY

The benchmarking process took place in an organized manner so as to ensure that there is a clear comparison made between fixed-point and double implementations of the Mandelbrot set algorithm. The following steps were followed:

- 1) **Algorithm Understanding:** A review of the definition of the Mandelbrot set and the given pseudocode. Thus, ensuring a clear linkage between the algorithm processes and the target C administration.
- 2) **Environment Setup:**
 - STM32F0 development board used as the benchmark platform.
 - Code written in C using STM32CubeIDE with STM32 HAL library serving as an intermediary between the hardware and software.
 - Live expressions and debugger features enabled for an impartial reporting of variable measurements throughout execution.
- 3) **Global Variable Definition:** Defined image dimension constants, global checksum storage, and timing variables (`start_time`, `end_time`) to allow for ongoing measurement.
- 4) **Implementation of Two Variants:**
 - **Fixed-Point Arithmetic:** Scaled integer representation with factor 10^6 to avoid floating-point operations.
 - **Double Precision:** Used C `double` type for all coordinate and iterative calculations.

Both versions followed the same iteration limit (`MAX_ITER = 100`) for consistency.

- 5) **Execution Time Measurement:** Used `HAL_GetTick()` before and after function calls to obtain execution duration in milliseconds.
- 6) **Dataset Selection:** Benchmarked the algorithm for five square resolutions: 128×128 , 160×160 , 192×192 , 224×224 , and 256×256 .
- 7) **Data Collection:**
 - Recorded execution time and checksum values from the Live Expressions window.
 - Comparison of results to recorded Python data to verify credibility within $\pm 1\%$ checksum tolerances.

III. RESULTS AND DISCUSSION

TABLE I
MANDELBROT BENCHMARK RESULTS USING PYTHON CODE

Resolution	Time (ms)	Checksum
128×128	79.0	429384
160×160	122.0	669829
192×192	171.0	966024
224×224	232.0	1314999
256×256	302.0	1715812

TABLE II
MANDELBROT BENCHMARK RESULTS USING FIXED POINT ARITHMETIC AND 64 BIT

Resolution	Time (ms)	Checksum
128×128	122054	428633
160×160	190731	668929
192×192	275415	965164
224×224	375095	1313861
256×256	489857	1714439

TABLE III
MANDELBROT BENCHMARK RESULTS USING DOUBLE

Resolution	Time (ms)	Checksum
128×128	122868	428671
160×160	193827	668949
192×192	279387	964961
224×224	379433	1313775
256×256	493055	1714436

- 1) **Difference in checksum between STM32F0 and PC**
The checksum values have a small difference between one another due to rounding errors in the fixed-point implementation, as well as the precision and accuracy differences between double-precision floating point (on the PC) and the scaled integer rendering of the STM32F0.

- 2) **Accuracy and Performance** Double values offer greater accuracy as they store a larger range and greater precision. Fixed-point on the STM32F0 is faster in theory due to an absence of a hardware FPU, however this was trivial due 64-bit integer math and scaling which reduced this benefit.
- 3) **If only integers were used?** The algorithm needs incremental precision to precisely represent the Mandelbrot set. Using integers would reduce all fractional values to zero, thus causing the output to be incorrect.
- 4) **Will there be a difference if we use floats instead of doubles?** Yes. Floats have a smaller memory size of 4 bytes as well as lower precision as opposed to doubles (8 bytes and greater precision). On the STM32F0, both are computed in software so difference in speed is rendered irrelevant. However, on microcontrollers with a Floating Point Unit (FPU), floats are significantly faster.
- 5) **How does `max_iter` affect execution time?** Execution time increases at a steady rate with `max_iter`, as each pixel's loop can run that many iterations. Doubling `max_iter` roughly doubles computation time but improves detail in the Mandelbrot image.

IV. CONCLUSION

The STM32F0 microcontroller, operating without hardware floating-point support, experiences fairly long execution times for complex computations such as the Mandelbrot set calculation. Both fixed-point and double-precision floating-point algorithms produced similar checksum values, testifying the correctness of the algorithm within the recommended tolerances.

The difference in performance between the fixed-point arithmetic and double-precision floating-point was unexpectedly insignificant which could be due to the overhead of 64-bit operations on integers that were required with this experiment, which results in more instructions to compute 64-bit integers.

Execution time increased greatly with image resolution, agreeing with the increased computational workload.

The use of STM32CubeIDE's debugging and live expression features were effective for categorizing execution times and confirming intermediate results without consecutive communication.

In the future, the STM32F4 should be used to benchmark performance. It features a faster Cortex-M4 core, higher clock speeds (up to 168 MHz), and a hardware FPU for single-precision floats, as well as more memory. This would significantly improve Mandelbrot performance, especially for floating-point calculations.

V. AI CLAUSE

Artificial Intelligence was used in this practical. It provided explanations for concepts such as the Mandelbrot set and helped in optimisation of code for the experiment. AI was also used in the creation of this report as it assisted with grammatical errors and clearer communication.

```

/* SMTJOS022 X BHYEBR002 EEE3096S Practical 1B*/

/**
*****
* @file          : main.c
* @brief         : Main program body
*****
* @attention
*
* Copyright (c) 2025 STMicroelectronics.
* All rights reserved.
*
* This software is licensed under terms that can be found in the LICENSE
file
* in the root directory of this software component.
* If no LICENSE file comes with this software, it is provided AS-IS.
*
*****
*/

/* USER CODE END Header */

/* Includes -----
----*/

#include "main.h"

/* Private includes -----
----*/

/* USER CODE BEGIN Includes */
#include <stdint.h>
#include "stm32f0xx.h"
// #include <Lcd_stm32f0.c>

```

```

#include <stdlib.h>

/* USER CODE END Includes */

/* Private typedef -----
----*/

/* USER CODE BEGIN PTD */
#define MAX_ITER 100
#define SCALE 1000000
/* USER CODE END PTD */

/* Private define -----
----*/

/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----
----*/

/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----
----*/

/* USER CODE BEGIN PV */
//TODO: Define and initialise the global variables required

int dim[] = {128, 160, 192, 224, 256};
uint32_t start_time=0, end_time=0, execution_time=0;
uint64_t check_sum=0;
uint64_t checksums[5];
uint32_t exec_times[5];

```

```

/*
    start_time
    end_time
    execution_time
    checksum: should be uint64_t
    initial width and height maybe or you might opt for an array??
*/

/* USER CODE END PV */

/* Private function prototypes -----
----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
/* USER CODE BEGIN PFP */
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int
height, int max_iterations);
uint64_t calculate_mandelbrot_double(int width, int height, int
max_iterations);

/* USER CODE END PFP */

/* Private user code -----
----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */

```

```

int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the
    SysTick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    /* USER CODE BEGIN 2 */

    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);

    //start_time = HAL_Get_Tick();

```

```

    //check_sum = calculate_mandelbrot_fixed_point_arithmetic(dim[0],
dim[0], MAX_ITER);

    //end_time = HAL_GetTick();

    //execution_time = end_time - start_time;

const int num_sizes = sizeof(dim) / sizeof(dim[0]);

    // Arrays to hold results for viewing in debugger

    for (int i = 0; i < num_sizes; i++) {

        start_time = HAL_GetTick();

        check_sum = calculate_mandelbrot_fixed_point_arithmetic(dim[i],
dim[i], MAX_ITER);

        end_time = HAL_GetTick();

        execution_time = end_time - start_time;

        // Store results
        checksums[i] = check_sum;
        exec_times[i] = execution_time;

        // Toggle LED1 on to signal completion of this run
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);

        // HoLd LEDs on for 1 second
        HAL_Delay(1000);

```

```

        // Turn off LED1 before next run
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);
    }

    // Turn off LED0 after all runs done
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);

//TODO: Turn on LED 1 to signify the end of the operation

//TODO: Hold the LEDs on for a 1s delay

//TODO: Turn off the LEDs

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None

```



```

    */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Initializes the RCC Oscillators according to the specified
    parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                   |RCC_CLOCKTYPE_PCLK1;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
    {
        Error_Handler();
    }
}

```

```

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1, GPIO_PIN_RESET);

    /*Configure GPIO pins : PB0 PB1 */
    GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */
    /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

```

```

//TODO: Mandelbrot using variable type integers and fixed point
arithmetic

uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int
height, int max_iterations){
    uint64_t mandelbrot_sum = 0;

    //TODO: Complete the function implementation

    for (int y = 0; y < height-1; y++){
        for (int x = 0; x < width-1; x++){
            // Convert to fixed-point coordinates
            //  $x_0 = (x/width) * 3.5 - 2.5$ 
            int64_t x0 = ((int64_t)x * 3500000 / width) - 2500000;
            //  $y_0 = (y/height) * 2.0 - 1.0$ 
            int64_t y0 = ((int64_t)y * 2000000 / height) - 1000000;
            int64_t xi = 0, yi = 0;
            int iter = 0;

            while (iter < max_iterations){
                int64_t xi2 = (xi*xi)/SCALE;
                int64_t yi2 = (yi*yi)/SCALE;

                if (xi2 + yi2 > 4 * SCALE) {
                    break;
                }

                int64_t temp = xi2-yi2;
                yi = (2*xi*yi)/SCALE+y0;
                xi = temp + x0;
                iter++;
            }
            mandelbrot_sum = mandelbrot_sum + iter;
        }
    }
}

```

```

    }

    return mandelbrot_sum;

}

//TODO: Mandelbroat using variable type double
uint64_t calculate_mandelbrot_double(int width, int height, int
max_iterations){
    uint64_t mandelbrot_sum = 0;
    //TODO: Complete the function implementation
    for (int y = 0; y < height-1; y++){
        for (int x = 0; x < width-1; x++){
            // Convert to fixed-point coordinates
            //  $x_0 = (x/width) * 3.5 - 2.5$ 
            double x0 = ((double)x * 3.5 / width) - 2.5;
            //  $y_0 = (y/height) * 2.0 - 1.0$ 
            double y0 = ((double)y * 2.0 / height) - 1.0;
            double xi = 0, yi = 0;
            int iter = 0;

            while (iter< max_iterations && (xi*xi+yi*yi)<=4){

                double temp = xi*xi-yi*yi;
                yi = (2*xi*yi)+y0;
                xi = temp + x0;
                iter++;
            }
            mandelbrot_sum = mandelbrot_sum + iter;
        }
    }
}

```

```

        return mandelbrot_sum;
    }

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return
    state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */

```

```
/* User can add his own implementation to report the file name and line number,
```

```
ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
```

```
/* USER CODE END 6 */
```

```
}
```

```
#endif /* USE_FULL_ASSERT */
```